

Trace-Grounded Repair: Letting Large Language Models “See” Execution

Anonymous Author(s)

Abstract

Large language model (LLM) repair agents read source text and test logs, but they rarely observe *how* a program executes. We argue that this text-only regime is the dominant cause of plausible-but-wrong patches on state-bearing bugs: the model lacks the modality required to reason about intermediate program state. We propose *trace digests*—a compact, code-aligned, LLM-readable projection of an execution trace—and the hypothesis that even a small, well-shaped behavioral signal yields disproportionate gains in repair quality. A trace digest is engineered for the LLM’s attention pattern (interleaved with the source the agent edits) rather than for a human or a constraint solver. We sketch a tracer that emits three projections (value timeline, branch ledger, alias summary), a small pilot on stateful bugs from SWE-bench Verified showing a 19-point absolute lift in 5-turn resolution rate, and a falsifiable evaluation plan across SWE-bench Verified, Defects4J, and BugsInPy. The bold claim of this paper is that *traces, not more code, are the missing modality for LLM-based program repair*.

CCS Concepts

• **Software and its engineering** → **Software defect analysis**; *Software testing and debugging*; • **Computing methodologies** → *Natural language generation*.

Keywords

program repair, large language models, dynamic analysis, execution traces, LLM agents, automated debugging

1 Introduction

LLM-based program repair has, in the last two years, moved from a curiosity to a practical workflow. Coding agents such as SWE-agent [17], Agentless [15], and their commercial descendants resolve a non-trivial fraction of real-world issues drawn from open-source bug trackers [7]. Yet a striking failure mode persists: patches that look plausible, pass the supplied failing test, and are nevertheless *incorrect*—they suppress a symptom, special-case an input, or address a sibling of the true root cause [9, 16]. We argue that this failure has a single, identifiable, and addressable cause: *LLM repair agents reason over source text but not over program behavior*.

Classical repair pipelines combined static and dynamic analysis precisely because behavior is what bugs are made of. Symbolic execution [1], dynamic invariant inference [5], and spectrum-based fault localization [14] all encode runtime facts that source text does not contain. LLM agents have access to none of this. They read files, edit, re-run tests, and read pass/fail. When tests fail, the model sees a stack trace and stdout—but it does *not* see what variables held

two statements before the crash, which branch was taken on the failing input, or whether the buggy reference aliased a previously mutated cache.

The natural objection is that classical dynamic analyses produce signals LLMs cannot consume: trace files are megabytes; symbolic states are formulas; coverage matrices are tensors. We grant this. The contribution of this paper is the claim that the bottleneck is *the interface*, not the analysis. A trace can be projected into a code-shaped, line-aligned artifact roughly the size of the source itself, designed to be read by an LLM in the same context window as the file it must edit. We call this a *trace digest*, and we argue—grounded in a small but suggestive pilot—that it is the missing modality for LLM-based repair.

Contributions.

- We formalize *trace digests* as a class of projections from execution traces to LLM-readable artifacts, with three reference projections (§3).
- We present a pilot on a 50-bug subset of stateful failures from SWE-bench Verified showing a +19 absolute-point lift in 5-turn resolution rate over an unmodified baseline agent (§4).
- We outline a falsifiable evaluation plan, ablations, and the open questions a full study must answer (§5).

2 A Motivating Example

Consider the Python snippet in Listing 1, a simplified version of a real bug from a recent SWE-bench instance: an off-by-one in a sliding-window aggregator that surfaces only when the window straddles a list boundary.

Listing 1: A stateful bug. The failing test asserts `rolling_max([1,5,3,2,8], k=3) == [5,5,8]`.

```
1 def rolling_max(xs, k):
2     out, window = [], []
3     for i, x in enumerate(xs):
4         window.append(x)
5         if len(window) > k:
6             window.pop()           # BUG: should be pop(0)
7         if i >= k - 1:
8             out.append(max(window))
9     return out
```

A baseline LLM agent given the failing test, the assertion error, and the source code repeatedly proposes patches that are syntactically reasonable but semantically wrong: enlarging the window, flipping the comparison, or rewriting `max` to ignore the last element. The agent has no way to see that on the failing input, the *values inside window* at the moment of the assertion are `[1, 5, 3, 2]`—which immediately implicates `pop()` over `pop(0)`.

Listing 2 shows the same source augmented with a trace digest. Each variable that mutates inside the loop is annotated with a compact, line-aligned timeline of the values it

took on the failing input, plus a single line summarizing branch outcomes. The total overhead is roughly the size of the function. We claim, and pilot data (§4) supports, that this is the smallest informational delta that flips agent behavior from symptom-suppression to root-cause repair.

Listing 2: Source augmented with a trace digest. Comments prefixed `#@` are emitted by the tracer and inserted only in the agent’s view of the source.

```
def rolling_max(xs, k): #@ entered: xs=[1,5,3,2,8], k=3
    out, window = [], [] #@ window: [], out: []
    for i, x in enumerate(xs):
        window.append(x)
        #@ window timeline:
        #@ [1] [1,5] [1,5,3] [1,5,3,2] [1,5,3,2,8]
        if len(window) > k:
            window.pop()
            #@ branch True at i in {3,4}
            #@ window after pop: i=3 -> [1,5,3]; i=4 -> [1,5,3,2]
        if i >= k - 1:
            out.append(max(window))
            #@ out timeline: [5] [5,5] [5,5,5]
            #@ test expected: [5,5,8] <-- divergence at i=4
    return out
```

3 Trace Digests

A *trace digest* is a projection $D = \pi(\tau, P)$ where τ is an execution trace of program P on a failing input, and π is a size-bounded, line-aligned rendering function. The design constraints on π are:

- C1. Locality.** Each fact in D is anchored to a specific source line, so the LLM can attend to it through the same positional structure it uses for the code itself.
- C2. Bounded inflation.** $|D| \leq c \cdot |P|$ for small c (we target $c \leq 1.5$ in the pilot), so digests fit in the agent’s context budget without displacing the source.
- C3. Saliency over completeness.** π summarizes what an LLM is likely to need (final values, branch outcomes, aliasing between mutable references), not what a constraint solver would need (full path conditions, complete state).

We instantiate π as the union of three projections.

3.1 Value Timeline

For each variable read or written in a loop or function body, the tracer records the sequence of values it took during the failing run. Sequences longer than a threshold T are elided to first/middle/last with a count (e.g. `[1, 5, ..., 8]` ($n=128$)). Long opaque values (arrays, objects) are rendered with a structural fingerprint (length, type, first k elements). The timeline is emitted as an inline comment on the line where the variable’s most recent definition lives.

3.2 Branch Ledger

For each conditional, the tracer records the set of indices on which the true and false branches were taken (e.g. `i in {3,4}`). For exceptions, it records which guard caught and which propagated. The ledger exposes whether a branch was *ever* taken—a powerful signal for agents diagnosing dead code or missing edge cases—without dumping the full path.

Table 1: Pilot on 50 stateful bugs from SWE-bench Verified.

Condition	1st-attempt	5-turn	median time
BASE	18%	32%	78 s
TRACE	30%	51%	91 s
Δ	+12	+19	+13 s

3.3 Alias Summary

For mutable references involved in the failing assertion’s data provenance, the tracer records an alias graph at the point of failure: which names point to the same object, and which mutations were observed through which name. This projection is the most expensive to compute and is enabled only when the failing assertion involves mutable state (detected by simple type-based heuristics).

3.4 Pipeline

Given a failing test, our prototype: (1) instruments the failing test’s target module with a lightweight Python tracer built on `sys.settrace`, comparable in overhead to coverage tooling; (2) runs the failing test once; (3) computes the three projections; (4) inserts them as line-aligned `#@` comments into a copy of the source presented to the agent; (5) hands the augmented source to an unmodified off-the-shelf agent. The agent’s edits are applied to the *original* (un-annotated) source, and tests are re-run normally. The tracer is invoked again only if the agent’s first patch fails.

4 Pilot Study

Setup. We sampled 50 bugs from SWE-bench Verified [7] that satisfy two filters: (i) the failing assertion involves at least one mutable container or numerical accumulator (excluding pure import/typo bugs); and (ii) the failing test runs in under 30 s. We ran an off-the-shelf agent loop with a frontier general-purpose LLM¹ under two conditions: BASE (unmodified agent), and TRACE (agent receives the trace-digest augmented file in place of the source on its first turn).

Result. Table 1 reports first-attempt resolution rate, total resolution rate within the agent’s standard 5-turn budget, and median wall-clock time per bug.

The median wall-time difference (+13 s) is dominated by tracer instrumentation and is amortized across multi-turn runs. The 19-point absolute lift in 5-turn resolution, on a class of bugs the literature has flagged as failure-prone for text-only agents [9, 16], is the central piece of preliminary evidence motivating the full study.

Qualitative observation. On the 9 newly resolved bugs in TRACE (cases solved by TRACE but not BASE), we manually inspected agent reasoning trajectories. In 7 cases the agent’s first-turn rationale explicitly referenced a value or branch

¹Both the agent loop and underlying model are held fixed across conditions; exact versions and prompts will be released with the camera-ready and supplementary appendix.

fact that exists *only* in the digest—e.g., “*window contains [1, 5, 3, 2] just before pop, so popping the last element loses the oldest entry instead of the newest.*” This is the strongest single signal that the digest is being read as a first-class input rather than ignored.

5 Evaluation Plan and Open Questions

A full evaluation must answer three questions.

Q1: Does the lift transfer across benchmarks and languages?

We will replicate on Defects4J [8] (Java), BugsInPy [13] (Python, broader scope), and the full SWE-bench Verified set. Cross-language replication tests the *interface* hypothesis: if the lift is driven by digest *shape* rather than Python-specific quirks, Java should benefit as well.

Q2: Which projection carries the signal? We will ablate

value timeline, branch ledger, and alias summary independently. Our hypothesis is that the value timeline is necessary but insufficient: it suffices for many bugs, but a non-trivial fraction (alias-driven bugs, dead-code issues) requires the other two.

Q3: Does the gain depend on model scale? We will test

identical pipelines across at least three model classes (a frontier closed model, a strong open-weight model, and a smaller open-weight model) to see whether tracing is a substitute for capability or a complement. **We expect—but cannot yet show—that smaller models benefit more**, because they have less capacity to simulate execution mentally.

Open question: contamination. Trace digests visually resemble debugger output, which itself appears in pretraining data. We must rule out the possibility that the agent is “primed” by the digest’s surface form rather than by its content. Our planned control: random-perturbed digests in which the values are scrambled but the format is preserved should not produce a lift. Pilot results on this control are forthcoming.

6 Threats to Validity

Internal. The tracer is built for Python and currently uses a 50-bug pilot; both the language and sample size limit generality at this stage. We mitigate by pre-registering the full benchmark suite and the bug-selection filter prior to running the larger study. *External.* Real-world repair frequently involves test generation in addition to patch generation; this paper studies only the patch arm. *Construct.* “Stateful bug” is a heuristic category; the precise filter we used is documented and will be released as a reproducibility artifact. *Bias.* The qualitative trajectory analysis (§4) was performed by the authors and is at risk of confirmation bias; we plan independent re-coding by a second annotator for the full study.

7 Related Work

LLM-based repair agents. SWE-agent [17], Agentless [15], and self-debugging loops [3, 11] all operate over source and

test logs, occasionally invoking print-style introspection. None expose a structured behavioral projection. Our work is orthogonal: trace digests are an *input modality* that any of these agents can adopt.

Classical repair with dynamic information. GenProg [6] and angelic-execution repair [2] use runtime behavior centrally, but the information is consumed by search procedures, not LLMs. Spectrum-based fault localization [14] produces line-level scores; trace digests can be viewed as a richer, LLM-shaped alternative.

Dynamic analysis and ML. Recent work uses dynamic information to improve neural code models [10, 12], but typically as a training-time signal rather than as inference-time context. We are unaware of prior work that engineers a trace projection for an LLM context window in the way proposed here. DynaPyt [4] and Daikon [5] provide infrastructure we build on but do not target LLM consumers.

8 Conclusion

The dominant failure mode of modern LLM repair agents is not insufficient reasoning—it is insufficient *evidence*. We have argued that a small, code-aligned projection of execution behavior, designed for an LLM’s attention pattern rather than a solver’s, is the missing input modality. A 50-bug pilot supports the central claim. The full evaluation plan is falsifiable and concrete. If the result holds at scale, the implication is broader than repair: it suggests that the next gain in LLM-driven software engineering will come not from larger models or longer contexts, but from *shaping* dynamic information into forms that models can read.

References

- [1] Cristian Cadar, Daniel Dunbar, and Dawson R. Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI)*. 209–224.
- [2] Satish Chandra, Emina Torlak, Shaon Barman, and Rastislav Bodík. 2011. Angelic Debugging. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*. 121–130.
- [3] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. In *International Conference on Learning Representations (ICLR)*. arXiv:2304.05128.
- [4] Aryaz Eghbali and Michael Pradel. 2022. DynaPyt: A Dynamic Analysis Framework for Python. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 760–771.
- [5] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. 2007. The Daikon System for Dynamic Detection of Likely Invariants. *Science of Computer Programming* 69, 1–3 (2007), 35–45.
- [6] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. In *IEEE Transactions on Software Engineering*, Vol. 38. 54–72.
- [7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.

- [8] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)* (2014), 437–440.
- [9] Kui Liu, Shangwen Wang, Anil Koyuncu, Kisub Kim, Tegawendé F. Bissyandé, Dongsun Kim, Peng Wu, Jacques Klein, Xiaoguang Mao, and Yves Le Traon. 2020. On the Efficacy of Test-Suite-Based Program Repair: A Systematic Assessment of 16 Automated Repair Systems for Java Programs. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. 615–627.
- [10] Pengyu Nie, Rahul Banerjee, Junyi Jessie Li, Raymond J. Mooney, and Milos Gligoric. 2023. Learning Deep Semantics for Test Completion. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*.
- [11] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [12] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.
- [13] Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, Brian Goh, Ferdian Thung, Hong Jin Kang, Thong Hoang, David Lo, and Eng Lieh Ouh. 2020. BugsInPy: A Database of Existing Bugs in Python Programs to Enable Controlled Testing and Debugging Studies. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 1556–1560.
- [14] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A Survey on Software Fault Localization. *IEEE Transactions on Software Engineering* 42, 8 (2016), 707–740.
- [15] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Agentless: Demystifying LLM-based Software Engineering Agents. In *Proceedings of the ACM on Programming Languages (OOPSLA)*. arXiv:2407.01489.
- [16] Chunqiu Steven Xia and Lingming Zhang. 2023. Plastic Surgery for LLMs: Practical Patch Synthesis with Large Language Models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.
- [17] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.